

CIF — Channel Interface: block quick-reference

AXI4 front end → segment/hash → ROB → CDC → MCC. Left = host/AXI, right = memory-controller core.

Host & AXI ports

AXI_INTERCONNECT AXI crossbar — Routes the SoC masters onto the four CIF slave ports.

AXI4IP0..3 AXI4 IP port (slave) — One AXI4 slave per port; terminates the five AXI channels for that port.

Per-port AXI channels

AR_CH Read Address channel — Accepts read address/len/id (AR*), hands the request downstream.

AW_CH Write Address channel — Accepts write address/len/id (AW*).

W_CH Write Data channel — Accepts write beats + strobes (W*).

R_CH Read Data channel — Returns read beats + RRESP/RLAST to the master.

BRESP_CH Write Response (B) channel — Returns the write completion BRESP/BID.

Per-port request FIFOs

AR_REQ_FIFO read-address request FIFO — Buffers accepted AR requests so ARREADY can decouple from admission.

AW_REQ_FIFO write-address request FIFO — Same for AW; holds until paired with data.

W_BURST_FIFO write-data burst descriptor FIFO — Holds the descriptor (start_ptr + beat count) into WD_SRAM — not the W beats themselves.

Arbitration & validation

PORT_ARB_R read port arbiter — Picks one of four ports' read streams (independent from writes).

PORT_ARB_W write port arbiter — Picks one of four ports' write streams.

WR_PAIR write pair gate — Pairs an AW address with its W-data descriptor (start_ptr+count) before write admission.

REQ_VAL_R read request validator — Checks alignment/len/legality; computes segment count n_segs.

REQ_VAL_W write request validator — Same legality/segmentation check for writes.

ERR_HDLR_R read error handler — Flags DECERR/SLVERR; allocates + marks the ROB entry retire-eligible.

ERR_HDLR_W write error handler — Same error path for writes.

Segmentation & packet form

BURST_SPLIT_R read burst splitter — Cuts an AXI burst into row_size segments (one packet each).

BURST_SPLIT_W write burst splitter — Splits the write burst; maps each segment to a WD_SRAM slot.

PKT_FORM_R read packet former — Builds the per-segment request packet sent to MCC ({rob_index, dbuf_addr, daddr, beats}).

PKT_FORM_W write packet former — Builds the write packet ({rob_index, wd_slot, daddr, beats}).

R_ADR_HASH read address hash — XOR-hashes the row + BG + bank bits into DRAM coords; column left un-hashed.

W_ADR_HASH write address hash — Same — hashes row/BG/bank only, not column.

Reorder buffers

ROB_RD read reorder buffer — Tracks each in-flight read: start_ptr, exp_pkts, bitmap, status; retires when the bitmap is full.

ROB_WR write reorder buffer — Tracks in-flight writes; SLOT_FIFO+RETIRE_LOGIC restore AXID order.

Data SRAM & managers

RD_SRAM read data buffer — 512b/line store; MCC writes returned read data here at dbuf_addr. Dual-domain (MC + CIF ports).

WD_SRAM write data buffer — Holds write beats; MCC reads them at wd_slot to drive DFI.

RD_SRAM_MGR read buffer manager — Allocates a contiguous dbuf range at ROB alloc; frees it on merge/drain.

WR_SRAM_MGR write buffer manager — Allocates a per-burst slot-vector on arrival; frees per packet on completion.

Response path

MERGE_LOGIC read merge/serialize — Waits for all packets (bitmap full), drains RD_SRAM in segment order to R_CH, frees the range.

BRESP_GEN B-response generator — Forms the AXI write response from the ROB_WR completion.

RESP_DISP response dispatcher — 1-of-4 port demux routing B responses back to the right BRESP_CH.

Config (APB / CSR)

APB_SLAVE APB config slave — Host register access into the CSR block.

CSR control/status registers — Configuration + status word file.

CFG_REG general config — Top-level enable/mode bits.

ADR_CFG address-map config — Field widths/positions for the address decode.

GEOM_CFG DRAM geometry config — Ranks/BG/banks/rows/cols and row_size.

HASH_CFG hash config — XOR-tap selection for the address hash.

CDC — CIF » MCC

CH_R_PKT_FIFO read packet FIFO — Per-channel staging of formed read packets before the async crossing.

CH_W_PKT_FIFO write packet FIFO — Same staging for write packets.

CH_R_REQ_AFIFO read request async FIFO — CIF→MCC read-request clock-domain crossing.

CH_W_REQ_AFIFO write request async FIFO — CIF→MCC write-request crossing.

CH_R_RESP_AFIFO read response async FIFO — MCC→CIF read completion {rob_index,pkt_num,status} crossing.

CH_W_RESP_AFIFO write response async FIFO — MCC→CIF write completion crossing.

Downstream

MCC Memory Controller Core — The scheduler + DFI side: consumes packets, drives DRAM, returns data to RD_SRAM and tagged completions. (Separate design.)